

📘 05차시 양자화 원리와 bitsandbytes 설정

교육시간: 1일차 15:00–16:00 / 60분 | **문서 기준일:** 2026-11-04

01 | 개요·시간 배분

교육시간: 1일차 15:00–16:00 / 60분. 앞 차시 LoRA(로라, 저랭크 적응)에서 학습 매개변수 수를 줄였다면, 이번 차시는 원본 가중치를 낮은 비트 수로 저장하는 양자화(Quantization, 퀀타이제이션)를 다룬다. 0–10분: 메모리 문제와 숫자 표현 / 10–20분: 양자화 원리 / 20–30분: NF4와 QLoRA 연결 / 30–45분: Python 실습 / 45–55분: 16비트·4비트 비교 / 55–60분: 점검. 성공 기준: 비트 수별 이론 메모리 계산, CUDA 유무 확인, 지원 GPU에서 4비트 모델 로딩 및 추론, 측정값과 이론값 차이 설명. 실습 모델 Qwen/Qwen2.5-0.5B-Instruct는 교안 제작상 선택한 예시이며 과정안내서가 특정한 모델은 아니다. 학습 범위: 이 차시의 4비트 로딩은 추론 실습이다. 실제 QLoRA 학습은 10차시에서 수행한다.

---

02 | 배경·선수지식

모델 가중치는 일반적으로 부동소수점으로 표현한다. FP32는 가중치 1개당 32비트, FP16/BF16은 16비트이다. 저장 비트 수를 줄이면 가중치 저장 공간을 줄일 수 있으나, 정밀도와 연산 경로가 달라진다. 04차시 LoRA는 원본 가중치를 고정하고 작은 A·B 행렬을 학습했다. 양자화는 그 고정 가중치 자체를 압축 표현하는 방법이다. QLoRA는 4비트 양자화된 원본 가중치 위에 LoRA 어댑터를 학습하는 조합이다. 이론 예시: 5억 매개변수 × 16비트 ÷ 8 = 1.0GB; × 4비트 ÷ 8 = 0.25GB. 이는 가중치 원시 저장량이며 실제 VRAM 요구량이 아니다. 확인 질문: 4비트로 바꾸면 모델 전체 메모리가 반드시 1/4이 될까? 아니다. 스케일·제로포인트·메타데이터·활성값·KV 캐시·CUDA 런타임이 추가된다.

03 | 양자화 핵심 원리

양자화는 연속적인 값을 제한된 표현 단계에 대응시키는 작업이다. 대칭 균일 양자화의 개념 예시는 q = round(x / s), 복원 근삿값은 x_hat = s × q이다. s는 scale(스케일, 양자화 단위)이다. 실제 라이브러리는 블록별 통계·비균일 코드북을 사용하기도 한다. 예: x=0.73, s=0.1이면 q=7, x_hat=0.7로 약 0.03의 오차가 생긴다. 이 오차가 계층마다 누적될 수 있어 정확도 측정이 필요하다. NF4(NormalFloat 4, 정규분포 가중치용 4비트 형식)는 단순 균일 정수 양자화와 다르며, 정규분포를 가정한 비균일 코드북을 사용한다. Double quantization(더블 퀀타이제이션, 이중 양자화)은 양자화 스케일 정보까지 추가로 압축한다. compute dtype(컴퓨트 데이터 타입, 계산 정밀도)은 가중치 저장 비트 수와 별개로 FP16/BF16을 선택할 수 있다. 주의: 4비트 저장과 4비트 산술 연산은 동일한 의미가 아니다.

04 | 구조도·메모리 흐름

원본 모델: [가중치 FP16] → [선형층 계산] → [출력] 양자화 모델: [가중치 NF4 4bit + 스케일 정보] → [연산 시 적절한 정밀도로 복원/계산] → [출력] QLoRA 학습(10차시): [4bit 원본 W: 동결] ─┐ ├→ [합산 출력] → 손실 → [LoRA A/B만 갱신] [LoRA A/B: 학습] ──────┘ 메모리 구성: GPU VRAM = 양자화 가중치 + 양자화 메타데이터 + 활성값 + KV 캐시 + CUDA/프레임워크 할당 등. 동일한 문장·동일한 생성 설정으로 비교하더라도 부동소수점 계산·커널 차이로 출력이 달라질 수 있다. 교육용 계산: 500M 파라미터의 이론 가중치 저장량은 FP32 2.0GB, FP16 1.0GB, INT8 0.5GB, 4bit 0.25GB이다.

05 | Ubuntu 준비·설치

1. 기존 Python 3.11 가상환경을 활성화한다. `python3.11 -m venv .venv` 는 처음 한 번만 수행하고, `source .venv/bin/activate` 로 활성화한다.
2. `python -m pip install --upgrade pip` 로 패키지 설치 도구를 갱신한다.
3. PyTorch는 GPU 드라이버/CUDA에 맞는 공식 설치 선택기를 사용한다. GPU가 없는 환경에서도 이론 메모리 계산 실습은 가능하다.
4. `python -m pip install -U transformers accelerate bitsandbytes` 는 모델 실행·장치 배치·양자화 지원 패키지를 설치한다. torch는 설치환경에 맞춰 별도 확인한다.
5. `nvidia-smi` 는 NVIDIA 드라이버/GPU 상태를 확인한다. `python -c "import torch; print(torch.cuda.is_available())"` 는 PyTorch CUDA 접근 여부를 출력한다.
6. `python lesson05_memory_math.py` 는 GPU 없이 실행할 수 있다.
7. `python lesson05_quant_infer.py --mode 4bit` 는 CUDA 지원 GPU와 호환되는 패키지 환경에서 실행한다. 설치 버전은 배포일에 따라 달라지므로 실행 시 출력된 버전을 기록하고, 호환되지 않으면 공식 설치 안내를 먼저 확인한다.

---

06 | 실습 코드 — 핵심 설정

실습 파일 `lesson05_quant_infer.py`에는 모델 ID, CUDA 사전 검사, 버전 출력, 4비트 설정, 추론 및 최대 CUDA 메모리 출력이 들어 있다. 핵심 코드:

```
config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, quantization_config=config, device_map="auto"
)
```

`load_in_4bit`는 4비트 로딩, `nf4`는 코드북 형식, `compute_dtype`은 계산 정밀도, `double_quant`는 양자화 상수 추가 압축을 뜻한다. 전체 실행 코드는 동봉한 독립 `.py` 파일에 제공한다. 16비트 비교 모드도 포함되어 있다.

---

07 | 코드 해설·검증 지표

`torch.cuda.is_available()` 가 False면 GPU 추론을 강행하지 않고 종료한다. 이는 CUDA 없는 환경에서 예외가 발생하는 것을 막는다. `AutoTokenizer.from_pretrained()` 는 토크나이저를 로딩하고 `apply_chat_template()` 는 채팅 형식 입력을 토큰으로 변환한다. `model.generate(..., do_sample=False, max_new_tokens=64)` 는 최대 64개의 새 토큰을 생성한다. 출력에서 입력 길이만큼 잘라 새로 생성된 부분만 디코딩한다. `torch.cuda.max_memory_allocated()` 는 PyTorch가 추적하는 최대 할당량, `max_memory_reserved()` 는 캐싱 할당자를 포함한 최대 예약량이다. 시스템 전체 GPU 사용량이나 디스크 모델 크기와 동일하지 않다. 비교 시 같은 GPU, 동일 모델, 동일 질문, 동일 토큰 수를 사용한다. 모델 로딩 전 측정 상태 초기화와 별도 프로세스 실행으로 조건을 통제한다. 예상 출력 형식: `CUDA peak allocated MiB: <환경별 값>` ; 실제 수치는 하드웨어와 버전에 따라 다르므로 교안에 확정 숫자를 쓰지 않는다.

---

08 | 따라하기 실습 — 15분

실습 A(3분): `python lesson05_memory_math.py`를 실행한다. 500M 가정에서 32/16/8/4비트의 이론 GB를 기록한다. 실습 B(2분): `nvidia-smi`와 `python -c "import torch; print(torch.cuda.is_available())"`를 실행하여 GPU 준비 상태를 기록한다. 실습 C(5분): 지원 GPU에서 `python lesson05_quant_infer.py --mode 16bit`를 실행한다. 모델 다운로드가 필요하며 네트워크·디스크 공간을 확보한다. 실습 D(5분): 별도 프로세스로 `python lesson05_quant_infer.py --mode 4bit`를 실행한다. 두 실행의 peak allocated/reserved를 표로 비교한다. 기록표: 모드 | 모델 ID | torch/transformers/bitsandbytes 버전 | peak allocated(MiB) | peak reserved(MiB) | 응답 텍스트. CUDA가 없는 환경은 A/B를 필수 완료로 인정하고, C/D는 지원 GPU에서 재실행한다. 메모리 절감 효과는 예상이 아닌 실제 측정값으로 판단한다.

---

09 | 오류 대응·확장

오류 1: CUDA False → 드라이버·PyTorch 빌드·GPU 할당을 확인한다. CPU에서 4비트 bitsandbytes 실습을 무조건 지원한다고 가정하지 않는다. 오류 2: bitsandbytes 백엔드 오류 → OS·GPU 아키텍처·CUDA/PyTorch·bitsandbytes 호환표를 확인한다. 임의의 오래된 버전을 무조건 고정하지 않는다. 오류 3: CUDA out of memory → 다른 GPU 프로세스 종료, 더 작은 모델 선택, max_new_tokens 축소, 새 프로세스 실행을 고려한다. 오류 4: 모델 다운로드 실패 → 네트워크·모델 ID·접근 권한·캐시 공간을 확인한다. 오류 5: 16비트 모드가 4비트보다 응답이 다름 → 정밀도 차이와 계산 커널 차이일 수 있다. 정확도 우열은 여러 평가 질문으로 판단한다. 확장: 8비트 로딩은 `load_in_8bit=True` 로 별도 실험할 수 있으나 동일 BitsAndBytesConfig에 4비트·8비트를 동시에 켜지 않는다. 4비트 양자화와 LoRA 학습은 구별한다.

10 | 평가·정리·공식 참고

평가 ① 5억 매개변수의 FP16 원시 가중치 저장량은? 정답: 약 1.0GB(십진). 평가 ② NF4가 단순 균일 INT4와 같은가? 정답: 아니다. 비균일 코드북을 사용하는 4비트 표현이다. 평가 ③ 4비트 로딩만으로 QLoRA 학습이 완료되는가? 정답: 아니다. LoRA 어댑터 구성·학습 데이터·훈련 과정이 필요하다. 평가 ④ peak allocated와 nvidia-smi 사용량은 반드시 같은가? 정답: 아니다. 측정 범위가 다르다. 완료 체크: 이론 저장량 계산 □ / CUDA 확인 □ / 4비트 설정 의미 설명 □ / GPU 지원 시 추론 □ / 측정값 기록 □. 다음 차시: 06차시 학습 전후 비교 테스트. 실제로는 학습된 어댑터가 있어야 전후 비교가 가능하므로 학습 전·후 평가 절차와 미학습 기준선을 명확히 구분한다. 공식 참고(확인일 2026-11-04, 링크 제공; 본 실행에서 온라인 실시간 검증은 미완료): Hugging Face Transformers bitsandbytes: https://huggingface.co/docs/transformers/quantization/bitsandbytes Hugging Face PEFT: https://huggingface.co/docs/peft/index PyTorch CUDA memory: https://pytorch.org/docs/stable/notes/cuda.html PyTorch 설치 선택기: https://pytorch.org/get-started/locally/

동봉 실습 파일

`lesson05_memory_math.py` : GPU 없이 저장량 계산

`lesson05_quant_infer.py` : CUDA GPU에서 4비트·16비트 추론 비교

적대적 검증·개선 기록 (2026-11-05)

반영: 기존 코드·교안의 핵심 구조를 보존했습니다. 실제 GPU 실험은 별도 실행 검증이 필요합니다.

검증 구분: 코드 구문·입력 데이터·PDF 및 HTML 생성과 검증은 수행했으나 실제 모델 다운로드·GPU 학습·추론 품질·Ollama 배포는 별도의 실물 검증이 필요합니다.

재현성: 교육용 5스텝 학습의 손실 감소나 정확도 향상을 보장하지 않습니다. 모델 ID, 데이터 버전, 장치·패키지 버전, 난수 시드, 생성 설정을 기록해야 합니다.

공식 근거: SFTTrainer, PEFT 연동, 템플릿, PEFT 모델 저장, 양자화, Ollama GGUF (2026-11-05 확인)